

数据结构

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	iii
第一章 哈希表	1
1.1 GNU gp_hash_table 与 cc_hash_table	1
第二章 并查集	2
2.1 并查集	2
2.1.1 并查集	2
2.1.2 可撤销并查集	3
第三章 树状数组	4
3.1 树状数组与第 k 小	4
第四章 线段树	5
4.1 线段树基础	5
4.1.1 通用懒标记线段树与边界搜索	5
4.1.2 通用线段树与左右边界搜索	8
4.1.3 区间加与区间和	12
4.1.4 线段树：区间乘、加、赋值与模意义求和	13
4.2 李超线段树	15
4.2.1 离散李超树与最优直线编号	15
第五章 二叉搜索树 & 平衡树	17
5.1 二叉搜索树 & 平衡树	17
5.1.1 GNU rope 与版本共享	17
5.1.2 PBDS 有序树与顺序统计	18
5.2 Treap	18
5.2.1 FHQ Treap：可重集合与顺序统计	18
5.2.2 隐式 Treap：序列翻转、加值与区间和	21
5.2.3 状态序列 Treap：分组区间 GCD	24
5.3 Splay 树	27
5.3.1 Splay：可重集合与顺序统计	27
第六章 可持久化数据结构	32
6.1 可持久化线段树	32
6.1.1 主席树：静态区间第 k 小	32
6.1.2 可持久化数组：历史版本分叉与单点修改	33
6.1.3 可持久化区间加与版本拼接	35
6.1.4 树上路径点权第 k 小	38
6.1.5 静态区间不同数计数（前驱位置主席树）	41
第七章 树套树	43
7.1 树状数组套权值线段树	43
7.1.1 动态区间第 k 小（树状数组套权值线段树）	43

第八章 动态树	46
8.1 Link Cut Tree	46
8.1.1 Link-cut tree: 动态森林与路径异或	46
第九章 配套知识与验证范围	50
9.1 可持久化区间拼接的懒标记	50
9.2 杭州 2023 K: 跨版本拼接应用	51
9.3 并查集模板题的接口范围	52
第一部分 跨类依赖	53
附录 A 数论	54
A.1 模算术简介	54
A.1.1 静态模整数	54

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 哈希表

1.1 GNU gp_hash_table 与 cc_hash_table

GNU libstdc++ 扩展, 包含 ext/pb_ds/assoc_container.hpp。gp_map<K,V> 为探测哈希表, cc_map<K,V> 为碰撞链接哈希表; K 为不超过 64 位的整数, 复用下方 hash64。两者支持 operator[]、find、insert、erase(key)、clear、size、empty; insert 不覆盖已有键, operator[] 对不存在的键插入默认值。只查询时用 find 判断 end。通常查改摊还 O(1), 最坏 O(n), 空间 O(n), 不支持排名或有序前驱。纯键值查改可先试 gp_map; 需要在插入/删除其他键后保留找到的元素引用时可考虑 cc_map, 其节点分配和链接有成本, 速度仍须按题实测。gp_map 修改后重新 find; cc_map 的点迭代器不等于可递增的遍历迭代器, 删除该元素、clear 或销毁后不能保留引用。默认策略不直接调用 resize, 也不照搬 unordered_map::reserve。hash64 使用进程内固定随机种子, 无符号回绕; 只针对整数键, 不保证已知种子下的最坏碰撞。历史 associative_array AC 仅覆盖 gp_map, cc_map 另行验证。

需要排名与有序前驱时, PBDS 平衡树见第 18 页。

```
1 struct hash64
2 {
3     static uint64_t mix(uint64_t x)
4     {
5         x += 0x9e3779b97f4a7c15ULL;
6         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
7         x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
8         return x ^ (x >> 31);
9     }
10
11     size_t operator()(uint64_t x) const
12     {
13         static const uint64_t seed =
14             chrono::steady_clock::now().time_since_epoch().count();
15         return mix(x + seed);
16     }
17 };
18
19 template <class K, class V> using gp_map = __gnu_pbds::gp_hash_table<K, V, hash64>;
20 template <class K, class V> using cc_map = __gnu_pbds::cc_hash_table<K, V, hash64>;
```

第二章 并查集

2.1 并查集

2.1.1 并查集

dsu(n), 点号 $0..n-1$, $n \geq 0$ 。find(x) 路径减半; same(x,y) 判断同集合; merge(x,y) 把 y 的根接到 x 的根, 已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组, 组内点号递增、组间按当前根编号排列; 可能压缩路径, 但不改变分组。fa、sz 可读, sz 仅根处有效; 不要外部改写。本版只做路径减半, 不按大小或秩合并; 单次 find 最坏 $O(n)$ 。空间 $O(n)$, groups 另需 $O(n)$ 输出与工作空间, 并执行 n 次 find。空结构可调用 groups, 不能查询顶点。

```
1 struct dsu
2 {
3     vector<int> fa, sz;
4
5     dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }
6
7     int find(int x)
8     {
9         while (x != fa[x]) x = fa[x] = fa[fa[x]];
10        return x;
11    }
12
13    bool same(int x, int y) { return find(x) == find(y); }
14
15    bool merge(int x, int y)
16    {
17        x = find(x), y = find(y);
18        if (x == y) return false;
19        sz[x] += sz[y], fa[y] = x;
20        return true;
21    }
22
23    int size(int x) { return sz[find(x)]; }
24
25    vector<vector<int>> groups()
26    {
27        vector<vector<int>> res(fa.size());
28        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
29        res.erase(
30            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
31            res.end());
32        return res;
33    }
34};
```

2.1.2 可撤销并查集

构造后 `snapshot()` 保存历史长度，`rollback(t)` 回退。禁止路径压缩；`find/merge` 为 $O(\log n)$ ，回退每次成功合并 $O(1)$ 。

```

1 struct RollbackDSU
2 {
3     vector<int> fa, siz;
4     vector<pair<int, int>> his;
5
6     RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }
7
8     int find(int x) const
9     {
10         while (x != fa[x]) x = fa[x];
11         return x;
12     }
13
14     int snapshot() const { return (int)his.size(); }
15
16     bool merge(int x, int y)
17     {
18         x = find(x);
19         y = find(y);
20         if (x == y) return false;
21         if (siz[x] < siz[y]) swap(x, y);
22         his.push_back({y, siz[x]});
23         fa[y] = x;
24         siz[x] += siz[y];
25         return true;
26     }
27
28     void rollback(int t)
29     {
30         assert(0 <= t && t <= snapshot());
31         while (snapshot() > t)
32         {
33             auto [y, s] = his.back();
34             his.pop_back();
35             siz[fa[y]] = s;
36             fa[y] = y;
37         }
38     }
39 };

```

第三章 树状数组

3.1 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。T 须能从 0 构造并支持 $+=$ 与减法；可直接使用 `ModInt` 维护模数下的区间和。 kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```

第四章 线段树

4.1 线段树基础

4.1.1 通用懒标记线段树与边界搜索

模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。 $0 \leq n \leq 2^{28}-1$ ，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 $O(\log n)$ ，构造 $O(n)$ 、操作/搜索 $O(\log n)$ 、all $O(1)$ ，空间 $O(n)$ ，假设运算和类型复制 $O(1)$ 。d/tag 及其他递归辅助方法仅供内部使用。

```
1 template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
2 struct lazy_segtree
3 {
4     int n;
5     vector<S> d;
6     vector<F> tag;
7
8     lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}
9
10    lazy_segtree(const vector<S> &a)
11        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
12    {
13        if (n) build(1, 0, n, a);
14    }
15
16    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18    void build(int p, int l, int r, const vector<S> &a)
19    {
20        if (r - l == 1)
21        {
22            d[p] = a[l];
23            return;
24        }
25        int m = (l + r) / 2;
26        build(p * 2, l, m, a);
27        build(p * 2 + 1, m, r, a);
28        pull(p);
29    }
```

```

31 void act(int p, F f)
32 {
33     d[p] = mapping(f, d[p]);
34     tag[p] = composition(f, tag[p]);
35 }
36
37 void push(int p)
38 {
39     act(p * 2, tag[p]);
40     act(p * 2 + 1, tag[p]);
41     tag[p] = id();
42 }
43
44 void set(int x, S v)
45 {
46     assert(0 <= x && x < n);
47     put(1, 0, n, x, v);
48 }
49
50 void put(int p, int l, int r, int x, S v)
51 {
52     if (r - l == 1)
53     {
54         d[p] = v;
55         tag[p] = id();
56         return;
57     }
58     push(p);
59     int m = (l + r) / 2;
60     if (x < m)
61         put(p * 2, l, m, x, v);
62     else
63         put(p * 2 + 1, m, r, x, v);
64     pull(p);
65 }
66
67 void apply(int l, int r, F f)
68 {
69     assert(0 <= l && l <= r && r <= n);
70     if (l < r) update(1, 0, n, l, r, f);
71 }
72
73 void update(int p, int l, int r, int a, int b, F f)
74 {
75     if (a <= l && r <= b)
76     {
77         act(p, f);
78         return;
79     }
80     push(p);
81     int m = (l + r) / 2;
82     if (a < m) update(p * 2, l, m, a, b, f);
83     if (m < b) update(p * 2 + 1, m, r, a, b, f);
84     pull(p);
85 }

```

```

87     S all() const { return d[1]; }
88
89     S get(int x)
90     {
91         assert(0 <= x && x < n);
92         return prod(x, x + 1);
93     }
94
95     S prod(int l, int r)
96     {
97         assert(0 <= l && l <= r && r <= n);
98         return l == r ? e() : query(1, 0, n, l, r);
99     }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }

```

```

133     template <class G> int min_left(int r, G ok)
134     {
135         assert(0 <= r && r <= n && ok(e()));
136         S s = e();
137         return r == 0 ? 0 : left(1, 0, n, r, s, ok);
138     }
139
140     template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
141     {
142         if (b <= l) return 0;
143         if (r <= b && ok(op(d[p], s)))
144         {
145             s = op(d[p], s);
146             return 0;
147         }
148         if (r - l == 1) return r;
149         push(p);
150         int m = (l + r) / 2;
151         int x = left(p * 2 + 1, m, r, b, s, ok);
152         return x != 0 ? x : left(p * 2, l, m, b, s, ok);
153     }
154 };

```

4.1.2 通用线段树与左右边界搜索

`segtree<S,op,e>(a)` 从 0-based vector 构建，或 `segtree(n)` 初始化为单位元。 $0 \leq n \leq 2^{29}$ ，实际受内存限制。`op(S,S)` 须满足结合律，`e()` 是双侧单位元；无需交换律。`set(p,x)`、`get(p)` 操作单点，`prod(l,r)` 返回半开区间 $[l,r)$ 从左到右的合并，空区间返回 `e`；`all()` 返回全区间结果。`max_right(l,ok)` 返回最大的 r 使 `ok(prod(l,r))` 为真；`min_left(r,ok)` 返回最小的 l 。要求 `ok(e())` 为真，`ok` 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。 $l=n$ 、 $r=0$ 支持空端点。构建 $O(n)$ ，单点/区间/边界搜索 $O(\log n)$ ，`all` $O(1)$ ，空间 $O(n)$ ，假设 `op`、`e`、`ok` 及 `S` 复制为 $O(1)$ 。`d`、`sz`、`pull` 为内部实现，不直接修改。函数复合时 `op(f,g)` 表示 `g(f(x))`，对应先执行左区间再执行右区间；`min_left` 不能照抄 `max_right` 的合并顺序。

```

1  template <class S, auto op, auto e> struct segtree
2  {
3      int n, sz = 1;
4      vector<S> d;
5
6      segtree(int n = 0) : segtree(vector<S>(n, e())) {}
7
8      segtree(const vector<S> &a) : n(a.size())
9      {
10         while (sz < n) sz <= 1;
11         d.assign(2 * sz, e());
12         copy(a.begin(), a.end(), d.begin() + sz);
13         for (int i = sz - 1; i; i--) pull(i);
14     }
15
16     void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18     void set(int p, S x)
19     {
20         assert(0 <= p && p < n);
21         d[p += sz] = x;

```

```
22     while (p >= 1) pull(p);
23 }
24
25 S get(int p) const
26 {
27     assert(0 <= p && p < n);
28     return d[p + sz];
29 }
30
31 S all() const { return d[1]; }
```

```

33  S prod(int l, int r) const
34  {
35      assert(0 <= l && l <= r && r <= n);
36      S a = e(), b = e();
37      for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
38      {
39          if (l & 1) a = op(a, d[l++]);
40          if (r & 1) b = op(d[--r], b);
41      }
42      return op(a, b);
43  }
44
45  template <class F> int max_right(int l, F ok) const
46  {
47      assert(0 <= l && l <= n && ok(e()));
48      if (l == n) return n;
49      S a = e();
50      l += sz;
51      do
52      {
53          while (!(l & 1)) l >>= 1;
54          if (!ok(op(a, d[l])))
55          {
56              while (l < sz)
57              {
58                  l *= 2;
59                  if (ok(op(a, d[l]))) a = op(a, d[l++]);
60              }
61              return l - sz;
62          }
63          a = op(a, d[l++]);
64      } while ((l & -1) != 1);
65      return n;
66  }

```

```
68     template <class F> int min_left(int r, F ok) const
69     {
70         assert(0 <= r && r <= n && ok(e()));
71         if (r == 0) return 0;
72         S a = e();
73         r += sz;
74         do
75         {
76             --r;
77             while (r > 1 && (r & 1)) r >>= 1;
78             if (!ok(op(d[r], a)))
79             {
80                 while (r < sz)
81                 {
82                     r = r * 2 + 1;
83                     if (ok(op(d[r], a))) a = op(d[r--], a);
84                 }
85                 return r + 1 - sz;
86             }
87             a = op(d[r], a);
88         } while ((r & -r) != r);
89         return 0;
90     }
91 };
```

4.1.3 区间加与区间和

初值全零； $\text{add}(l,r,v)$ 、 $\text{query}(l,r)$ ， $1 \leq l \leq r \leq n$ 。维护 sum 与加法 tag ；所有中间和与长度乘积须在 long long 范围内。单次 $O(\log n)$ 。

```

1 struct LazySeg
2 {
3     int n;
4     vector<long long> sum, tag;
5
6     LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4) { assert(n > 0); }
7
8     void apply(int p, int l, int r, long long v)
9     {
10         sum[p] += (r - l + 1) * v;
11         tag[p] += v;
12     }
13
14     void push(int p, int l, int r)
15     {
16         int m = (l + r) / 2;
17         apply(p * 2, l, m, tag[p]);
18         apply(p * 2 + 1, m + 1, r, tag[p]);
19         tag[p] = 0;
20     }
21
22     void add(int l, int r, long long v) { add(1, 1, n, l, r, v); }
23
24     void add(int p, int l, int r, int ql, int qr, long long v)
25     {
26         if (ql <= l && r <= qr)
27         {
28             apply(p, l, r, v);
29             return;
30         }
31         push(p, l, r);
32         int m = (l + r) / 2;
33         if (ql <= m) add(p * 2, l, m, ql, qr, v);
34         if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
35         sum[p] = sum[p * 2] + sum[p * 2 + 1];
36     }
37
38     long long query(int l, int r) { return query(1, 1, n, l, r); }
39
40     long long query(int p, int l, int r, int ql, int qr)
41     {
42         if (ql <= l && r <= qr) return sum[p];
43         push(p, l, r);
44         int m = (l + r) / 2;
45         long long ans = 0;
46         if (ql <= m) ans += query(p * 2, l, m, ql, qr);
47         if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
48         return ans;
49     }
50 };

```

4.1.4 线段树：区间乘、加、赋值与模意义求和

`AffineSegTree(a,mod)` 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。`update(l,r,mul,add)` 将区间元素 x 改为 $mul \cdot x + add$ ；乘法用 `(mul,0)`，加法用 `(1,add)`，赋值用 `(0,value)`。`query(l,r)` 返回区间和。 $1 \leq mod \leq INT_MAX$ ，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 `(m,b)` 后应用新标记 `(a,c)`，复合为 $(m \cdot a, b \cdot a + c)$ ，乘法取模后再相加。建树 $O(n)$ ，单次 $O(\log n)$ ，空间 $O(n)$ 。内部 `build`、`apply` 等函数不作为公共接口；动态版重建新对象。

```

1 struct AffineSegTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    vector<Node> t;
12
13    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
14    {
15        assert(n > 0 && mod > 0);
16        build(1, 1, n, a);
17    }
18
19    ll norm(ll x) const
20    {
21        x %= mod;
22        if (x < 0) x += mod;
23        return x;
24    }
25
26    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }
27
28    void build(int p, int l, int r, const vector<ll> &a)
29    {
30        t[p] = Node{};
31        if (l == r)
32        {
33            t[p].sum = norm(a[l - 1]);
34            return;
35        }
36        int m = (l + r) / 2;
37        build(p * 2, l, m, a);
38        build(p * 2 + 1, m + 1, r, a);
39        pull(p);
40    }
41
42    // New transform follows the old pending transform: x -> x*a+b.
43    void apply(int p, int len, ll a, ll b)
44    {
45        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
46        t[p].mul = t[p].mul * a % mod;
47        t[p].add = (t[p].add * a % mod + b) % mod;
48    }

```

```

49
50 void push(int p, int l, int r)
51 {
52     if (t[p].mul == 1 && t[p].add == 0) return;
53     int m = (l + r) / 2;
54     apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55     apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56     t[p].mul = 1;
57     t[p].add = 0;
58 }
59
60 void update(int l, int r, ll a, ll b)
61 {
62     assert(1 <= l && l <= r && r <= n);
63     update(1, 1, n, l, r, norm(a), norm(b));
64 }
65
66 void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67 {
68     if (ql <= l && r <= qr)
69     {
70         apply(p, r - l + 1, a, b);
71         return;
72     }
73     push(p, l, r);
74     int m = (l + r) / 2;
75     if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76     if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77     pull(p);
78 }
79
80 ll query(int l, int r)
81 {
82     assert(1 <= l && l <= r && r <= n);
83     return query(1, 1, n, l, r);
84 }
85
86 ll query(int p, int l, int r, int ql, int qr)
87 {
88     if (ql <= l && r <= qr) return t[p].sum;
89     push(p, l, r);
90     int m = (l + r) / 2;
91     ll answer = 0;
92     if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93     if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94     return answer % mod;
95 }
96 };

```

4.2 李超线段树

4.2.1 离散李超树与最优直线编号

构造传入所有可能查询的横坐标， $\text{add}(\{k,b,\text{id}\})$ 插入直线， $\text{query}(x)$ 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 `int128`， k 、 b 、 x 为有符号 64 位且中间值可表示。单次 $O(\log n)$ 。

```

1 struct LiChao
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Line
7     {
8         ll k = 0, b = 0;
9         int id = -1;
10
11         I value(ll x) const { return I(k) * x + b; }
12     };
13
14     struct Node
15     {
16         Line line;
17         int left = -1, right = -1;
18     };
19
20     vector<ll> xs;
21     vector<Node> a{Node{}};
22
23     LiChao(vector<ll> x) : xs(x)
24     {
25         sort(xs.begin(), xs.end());
26         xs.erase(unique(xs.begin(), xs.end()), xs.end());
27         assert(!xs.empty());
28     }
29
30     bool better(Line u, Line v, ll x) const
31     {
32         return v.id == -1 ||
33             (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
34     }
35
36     void add(Line line)
37     {
38         assert(line.id >= 0);
39         add(0, 0, (int)xs.size() - 1, line);
40     }
41
42     void add(int p, int l, int r, Line line)
43     {
44         int m = (l + r) / 2;
45         if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
46         if (l == r || line.id == -1) return;
47         bool left = better(line, a[p].line, xs[l]);
48         bool right = better(line, a[p].line, xs[r]);
49         if (!left && !right) return;

```

```

50     int next = left ? a[p].left : a[p].right;
51     if (next == -1)
52     {
53         next = (int)a.size();
54         a.push_back(Node{});
55         if (left)
56             a[p].left = next;
57         else
58             a[p].right = next;
59     }
60     if (left)
61         add(next, l, m, line);
62     else
63         add(next, m + 1, r, line);
64 }
65
66 pair<I, int> query(ll x) const
67 {
68     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
69     assert(idx < (int)xs.size() && xs[idx] == x);
70     int p = 0, l = 0, r = (int)xs.size() - 1;
71     Line best;
72     while (p != -1)
73     {
74         if (better(a[p].line, best, x)) best = a[p].line;
75         if (l == r) break;
76         int m = (l + r) / 2;
77         if (idx <= m)
78         {
79             p = a[p].left;
80             r = m;
81         }
82         else
83         {
84             p = a[p].right;
85             l = m + 1;
86         }
87     }
88     assert(best.id != -1);
89     return {best.value(x), best.id};
90 }
91 };

```

第五章 二叉搜索树 & 平衡树

5.1 二叉搜索树 & 平衡树

5.1.1 GNU rope 与版本共享

实际类型为 `__gnu_cxx::rope<T>`, 不是 `std::rope`; 它是 GNU 扩展序列, 包含 `ext/rope`。本工具链随后重新包含 `cassert`, 保留断言接口。`rp<int>`、`rp<char>` 表示序列, 默认空。`size()`、`empty()`、`push_back(x)`、`insert(p,x)`、`erase(p,len)`、`replace(p,x)`、`substr(p,len)` 直接沿用 `rope`; 位置为 0-based, `len` 是长度而非右端点。约定 $0 \leq p \leq \text{size}$, 删除/截取要求 $\text{len} \leq \text{size} - p$, 读写单点要求 $p < \text{size}$ 。`r[i]` 读取第 i 项; `substr` 返回新序列。`rp<T> b=a` 共享底层树, 随后通过容器接口修改 `b` 不影响 `a`, 可用 `vector<rp<int>` 保存分支版本。复制构造共享根为 $O(1)$; 随机访问沿树路径, 正常平衡时 $O(\log n)$ 。插入/删除/截取涉及路径、叶块复制和可能的再平衡, 不能把每次操作都写成严格最坏 $O(\log n)$; 释放最后引用可能递归回收整棵子树。长期保留所有版本的内存不能按单个序列 $O(n)$ 估计。不可缓存修改前的迭代器或使用可变代理跨版本写入。仅使用默认内存分配器及普通整数/字符值; 函数生成型 `rope` 不在本模板范围。可持久化队列已通过定长构造加头尾下标版本: Q 个零单元, 复制根与 l/r , 入队 `replace(r++,x)`, 出队读取 `[l++]`; 直接 `push_back/erase` 版本在针对数据上耗尽内存。线上 942 ms、591.26 MiB, 具体失败与通过范围见 `docs/ROPE-NOTES.md`。适合需要序列中间插删、截取和少量分支快照的场景; 只按下标读写通常使用 `vector` 更直接。需要区间和、翻转等懒标记聚合时使用隐式 `Treap`, `rope` 本身不提供这些接口。下方示例通过修改 `b` 演示 `a` 保持原版本。

只需键值查改而不需要有序排名时, `gp_hash_table` / `cc_hash_table` 及选型建议见第 1 页。

```
1 template <class T> using rp = __gnu_cxx::rope<T>;

2 rp<int> a;
3 a.push_back(3);
4 a.insert(0, 7);
5 rp<int> b = a;
6 b.replace(1, 9);
7 b.erase(0, 1);
8 auto c = a.substr(0, 1);
9 int x = a[1];
```

5.1.2 PBDS 有序树与顺序统计

GNU libstdc++ 扩展, 包含 `ext/pb_ds/assoc_container.hpp` 和 `ext/pb_ds/tree_policy.hpp`。 `ost<T>` 为去重有序集合; `insert(x)`, `erase(x)`, `find(x)`, `lower_bound(x)`, `upper_bound(x)` 沿用集合接口。 `order_of_key(x)` 为严格小于 `x` 的元素数, `find_by_order(k)` 返回 0-based 第 `k` 个的迭代器, 越界返回 `end`。默认 `less<T>`; 改用其他严格弱序比较器时, 排名和边界均按该比较器解释。单次 $O(\log n)$, 空间 $O(n)$, 假设比较为 $O(1)$ 。重复值使用 `ost<pair<T,int>`, 插入 `{x,id}` 且 `id` 全局唯一、 $1 \leq id < INT_MAX$ 。 `order_of_key({x,0})` 统计小于 `x`, `order_of_key({x,INT_MAX})` 统计不大于 `x`; `find_by_order(k)->first` 取值, 须先检查 `end`。删除一个 `x`: `it=lower_bound({x,0})`, 检查 `it!=end` 且 `it->first==x` 后 `erase(it)`。不可用 `less_equal` 模拟多重集合。前驱先检查迭代器是否为 `begin`, 再递减; 空集合和无解不得解引用 `end`。

只需键值查改而不需要有序排名时, `gp_hash_table` / `cc_hash_table` 及选型建议见第 1 页。

序列中间插删、截取与版本共享的 GNU rope 见第 17 页。

```
1 template <class T, class C = less<T>>
2 using ost = __gnu_pbds::tree<T,
3         __gnu_pbds::null_type,
4         C,
5         __gnu_pbds::rb_tree_tag,
6         __gnu_pbds::tree_order_statistics_node_update>;
```

5.2 Treap

5.2.1 FHQ Treap: 可重集合与顺序统计

`insert(x)` 插入一个副本, `erase(x)` 删除一个副本并返回是否存在; `rank(x)` 为严格小于 `x` 的元素数加一, `x` 不必存在; `kth(k)` 要求 $1 \leq k \leq \text{size}()$ 。 `prev/next` 为严格前驱/后继, 不存在时返回 `nullopt`。支持重复值、`LLONG_MIN` 和 `LLONG_MAX`, 分裂不使用 `x-1`。随机优先级下单次期望 $O(\log n)$, 并非确定性最坏界。节点池不回收已删除节点, 空间按累计插入次数计算; 可在构造时指定随机种子。

```
1 struct OrderedTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    mt19937_64 rng;
14    int root = 0;
15
16    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}
17
18    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }
19
20    int node(ll x)
21    {
22        int p = a.size();
23        a.push_back({0, 0, 1, x, rng()});
24        return p;
25    }
```

```

26
27 // left: val < x, or val <= x when equal is true. No x-1 overflow.
28 void split(int p, ll x, int &l, int &r, bool equal = false)
29 {
30     if (!p)
31     {
32         l = 0;
33         r = 0;
34         return;
35     }
36     if (a[p].val < x || (equal && a[p].val == x))
37     {
38         l = p;
39         split(a[p].r, x, a[p].r, r, equal);
40     }
41     else
42     {
43         r = p;
44         split(a[p].l, x, l, a[p].l, equal);
45     }
46     pull(p);
47 }
48
49 int merge(int l, int r)
50 {
51     if (!l || !r) return l ? l : r;
52     if (a[l].pri > a[r].pri)
53     {
54         a[l].r = merge(a[l].r, r);
55         pull(l);
56         return l;
57     }
58     a[r].l = merge(l, a[r].l);
59     pull(r);
60     return r;
61 }
62
63 void insert(ll x)
64 {
65     int p = node(x), l, r;
66     split(root, x, l, r);
67     root = merge(merge(l, p), r);
68 }
69
70 bool erase(ll x)
71 {
72     int l, mid, r;
73     split(root, x, l, mid);
74     split(mid, x, mid, r, true);
75     bool found = mid != 0;
76     if (mid) mid = merge(a[mid].l, a[mid].r);
77     root = merge(l, merge(mid, r));
78     return found;
79 }
80
81 int size() const { return a[root].siz; }

```

```

82
83 int less(ll x, bool equal = false) const
84 {
85     int p = root, ans = 0;
86     while (p)
87     {
88         if (a[p].val < x || (equal && a[p].val == x))
89         {
90             ans += a[a[p].l].siz + 1;
91             p = a[p].r;
92         }
93         else
94             p = a[p].l;
95     }
96     return ans;
97 }
98
99 int rank(ll x) const { return less(x) + 1; }
100
101 ll kth(int k) const
102 {
103     assert(1 <= k && k <= size());
104     int p = root;
105     while (true)
106     {
107         int left = a[a[p].l].siz;
108         if (k == left + 1) return a[p].val;
109         if (k <= left)
110             p = a[p].l;
111         else
112         {
113             k -= left + 1;
114             p = a[p].r;
115         }
116     }
117 }
118
119 optional<ll> prev(ll x) const
120 {
121     int k = less(x);
122     if (!k) return nullopt;
123     return kth(k);
124 }
125
126 optional<ll> next(ll x) const
127 {
128     int k = less(x, true);
129     if (k == size()) return nullopt;
130     return kth(k + 1);
131 }
132 };

```

5.2.2 隐式 Treap：序列翻转、加值与区间和

`insert(k,value)` 在前 k 个元素后插入， $0 \leq k \leq \text{size}()$ ；`erase(l,r)`、`reverse(l,r)`、`add(l,r,value)`、`query(l,r)` 均使用 1-based 非空闭区间。`values()` 返回当前完整序列， $O(n)$ ，使用递归中序遍历。其余操作在随机优先级下期望 $O(\log n)$ ；翻转与加法懒标记同时维护。所有值、和、标记及 `value × 长度` 中间结果必须落在 `long long` 范围内。删除不回收节点，内存按累计插入数计算。

```

1 struct SequenceTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0, sum = 0, tag = 0;
9         unsigned long long pri = 0;
10        bool rev = false;
11    };
12
13    vector<Node> a{Node{}};
14    mt19937_64 rng;
15    int root = 0;
16
17    SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}
18
19    int size() const { return a[root].siz; }
20
21    void pull(int p)
22    {
23        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
24        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
25    }
26
27    void apply_add(int p, ll x)
28    {
29        if (!p) return;
30        a[p].val += x;
31        a[p].sum += x * a[p].siz;
32        a[p].tag += x;
33    }
34
35    void apply_reverse(int p)
36    {
37        if (!p) return;
38        swap(a[p].l, a[p].r);
39        a[p].rev = !a[p].rev;
40    }
41
42    void push(int p)
43    {
44        if (a[p].rev)
45        {
46            apply_reverse(a[p].l);
47            apply_reverse(a[p].r);
48            a[p].rev = false;
49        }

```

```

50     if (a[p].tag)
51     {
52         apply_add(a[p].l, a[p].tag);
53         apply_add(a[p].r, a[p].tag);
54         a[p].tag = 0;
55     }
56 }
57
58 // Split into the first k elements and the rest.
59 void split(int p, int k, int &l, int &r)
60 {
61     if (!p)
62     {
63         l = 0;
64         r = 0;
65         return;
66     }
67     push(p);
68     int left = a[a[p].l].siz;
69     if (k > left)
70     {
71         l = p;
72         split(a[p].r, k - left - 1, a[p].r, r);
73     }
74     else
75     {
76         r = p;
77         split(a[p].l, k, l, a[p].l);
78     }
79     pull(p);
80 }
81
82 int merge(int l, int r)
83 {
84     if (!l || !r) return l ? l : r;
85     if (a[l].pri > a[r].pri)
86     {
87         push(l);
88         a[l].r = merge(a[l].r, r);
89         pull(l);
90         return l;
91     }
92     push(r);
93     a[r].l = merge(l, a[r].l);
94     pull(r);
95     return r;
96 }
97
98 // Insert after the first k elements, 0 <= k <= size().
99 void insert(int k, ll value)
100 {
101     assert(0 <= k && k <= size());
102     int p = a.size(), l, r;
103     a.push_back({0, 0, 1, value, value, 0, rng(), false});
104     split(root, k, l, r);
105     root = merge(merge(l, p), r);

```

```

106     }
107
108     // All range operations use 1-based inclusive intervals.
109     void erase(int l, int r)
110     {
111         assert(1 <= l && l <= r && r <= size());
112         int x, y, z;
113         split(root, r, y, z);
114         split(y, l - 1, x, y);
115         root = merge(x, z);
116     }
117
118     void reverse(int l, int r)
119     {
120         assert(1 <= l && l <= r && r <= size());
121         int x, y, z;
122         split(root, r, y, z);
123         split(y, l - 1, x, y);
124         apply_reverse(y);
125         root = merge(merge(x, y), z);
126     }
127
128     void add(int l, int r, ll value)
129     {
130         assert(1 <= l && l <= r && r <= size());
131         int x, y, z;
132         split(root, r, y, z);
133         split(y, l - 1, x, y);
134         apply_add(y, value);
135         root = merge(merge(x, y), z);
136     }
137
138     ll query(int l, int r)
139     {
140         assert(1 <= l && l <= r && r <= size());
141         int x, y, z;
142         split(root, r, y, z);
143         split(y, l - 1, x, y);
144         ll ans = a[y].sum;
145         root = merge(merge(x, y), z);
146         return ans;
147     }
148
149     void collect(int p, vector<ll> &out)
150     {
151         if (!p) return;
152         push(p);
153         collect(a[p].l, out);
154         out.push_back(a[p].val);

```

```

155     collect(a[p].r, out);
156 }
157
158 vector<ll> values()
159 {
160     vector<ll> answer;
161     collect(root, answer);
162     return answer;
163 }
164 };

```

5.2.3 状态序列 Treap：分组区间 GCD

build(v) 用数值与状态组成的序列替换当前内容。数值为无符号 64 位，状态为 0 或 1。insert(k,value,tag) 在前 k 项后插入。erase(l,r)、query(l,r,tag) 使用 1-based 非空闭区间；set(k,value) 修改值，toggle(k) 翻转状态。query 返回选中状态的 GCD，无对应元素为 nullopt，实际全零则返回 0。回收删除节点，空间按峰值长度计算。建树 $O(n)$ 次 GCD，随机优先级下单次期望 $O(\log n)$ 次 GCD；删除另需 $O(\text{删除长度})$ 回收。一次 GCD 为 $O(\log U)$ 。覆盖 kuangbin 3.7 的序列操作，尚无该原题 OJ 记录。建树后的聚合与删除节点回收均使用递归遍历；笛卡尔树的构造单调栈保留。

```

1 struct GcdSequenceTreap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    vector<int> recycled;
14    mt19937_64 rng;
15    int root = 0;
16
17    GcdSequenceTreap(U seed = 712367821) : rng(seed) {}
18
19    int size() const { return a[root].siz; }
20
21    int node(U value, int tag)
22    {
23        assert(tag == 0 || tag == 1);
24        int p;
25        if (recycled.empty())
26        {
27            p = a.size();
28            a.push_back(Node{});
29        }
30        else
31        {
32            p = recycled.back();
33            recycled.pop_back();
34            a[p] = Node{};
35        }

```

```

36     a[p].siz = 1;
37     a[p].val = value;
38     a[p].tag = tag;
39     a[p].cnt[tag] = 1;
40     a[p].g[tag] = value;
41     a[p].pri = rng();
42     return p;
43 }
44
45 void pull(int p)
46 {
47     int l = a[p].l, r = a[p].r;
48     a[p].siz = a[l].siz + a[r].siz + 1;
49     for (int t = 0; t < 2; t++)
50     {
51         a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
52         a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
53         if (a[p].tag == t) a[p].g[t] = gcd(a[p].g[t], a[p].val);
54     }
55 }
56
57 void split(int p, int k, int &l, int &r)
58 {
59     if (!p)
60     {
61         l = 0;
62         r = 0;
63         return;
64     }
65     int left = a[a[p].l].siz;
66     if (k > left)
67     {
68         l = p;
69         split(a[p].r, k - left - 1, a[p].r, r);
70     }
71     else
72     {
73         r = p;
74         split(a[p].l, k, l, a[p].l);
75     }
76     pull(p);
77 }
78
79 int merge(int l, int r)
80 {
81     if (!l || !r) return l ? l : r;
82     if (a[l].pri > a[r].pri)
83     {
84         a[l].r = merge(a[l].r, r);
85         pull(l);
86         return l;
87     }
88     a[r].l = merge(l, a[r].l);
89     pull(r);
90     return r;
91 }

```

```

92
93 void rebuild(int p)
94 {
95     if (!p) return;
96     rebuild(a[p].l);
97     rebuild(a[p].r);
98     pull(p);
99 }
100
101 void release(int p)
102 {
103     if (!p) return;
104     release(a[p].l);
105     release(a[p].r);
106     recycled.push_back(p);
107 }
108
109 // O(n) GCD calls; replaces the current sequence.
110 void build(const vector<pair<U, int>> &v)
111 {
112     a.assign(1, Node{});
113     recycled.clear();
114     root = 0;
115     vector<int> st;
116     for (auto [value, tag] : v)
117     {
118         int p = node(value, tag), last = 0;
119         while (!st.empty() && a[st.back()].pri < a[p].pri)
120         {
121             last = st.back();
122             st.pop_back();
123         }
124         a[p].l = last;
125         if (!st.empty()) a[st.back()].r = p;
126         st.push_back(p);
127     }
128     if (st.empty()) return;
129     root = st.front();
130     rebuild(root);
131 }
132
133 // Insert after the first k entries. Range operations below are 1-based.
134 void insert(int k, U value, int tag)
135 {
136     assert(0 <= k && k <= size());
137     int p = node(value, tag), l, r;
138     split(root, k, l, r);
139     root = merge(merge(l, p), r);
140 }
141
142 void erase(int l, int r)
143 {
144     assert(1 <= l && l <= r && r <= size());
145     int x, y, z;
146     split(root, r, y, z);
147     split(y, l - 1, x, y);

```

```

148     root = merge(x, z);
149     release(y);
150 }
151
152 void set(int k, U value)
153 {
154     assert(1 <= k && k <= size());
155     int x, y, z;
156     split(root, k, y, z);
157     split(y, k - 1, x, y);
158     a[y].val = value;
159     pull(y);
160     root = merge(merge(x, y), z);
161 }
162
163 void toggle(int k)
164 {
165     assert(1 <= k && k <= size());
166     int x, y, z;
167     split(root, k, y, z);
168     split(y, k - 1, x, y);
169     a[y].tag ^= 1;
170     pull(y);
171     root = merge(merge(x, y), z);
172 }
173
174 optional<U> query(int l, int r, int tag)
175 {
176     assert(1 <= l && l <= r && r <= size());
177     assert(tag == 0 || tag == 1);
178     int x, y, z;
179     split(root, r, y, z);
180     split(y, l - 1, x, y);
181     optional<U> answer;
182     if (a[y].cnt[tag]) answer = a[y].g[tag];
183     root = merge(merge(x, y), z);
184     return answer;
185 }
186 };

```

5.3 Splay 树

5.3.1 Splay：可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义，严格前驱后继不存在返回 nullopt。重复键计数存储；失败查询也伸展最后访问节点，单次均摊 $O(\log n)$ 。不回收节点，空间按累计新建节点数计算。支持 long long 全范围；不包含序列 Splay、虚子树信息或矩阵动态 DP。

```

1 struct OrderedSplay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0, cnt = 0, siz = 0;

```

```

8      ll val = 0;
9  };
10
11  vector<Node> a{Node{}};
12  int root = 0;
13
14  int size() const { return a[root].siz; }
15
16  void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }
17
18  int node(ll value, int parent)
19  {
20      int x = a.size();
21      a.push_back({{0, 0}, parent, 1, 1, value});
22      return x;
23  }
24
25  void rotate(int x)
26  {
27      int y = a[x].fa, z = a[y].fa;
28      int k = a[y].ch[1] == x;
29      int w = a[x].ch[k ^ 1];
30      if (z)
31          a[z].ch[a[z].ch[1] == y] = x;
32      else
33          root = x;
34      a[x].fa = z;
35      a[y].ch[k] = w;
36      if (w) a[w].fa = y;
37      a[x].ch[k ^ 1] = y;
38      a[y].fa = x;
39      pull(y);
40      pull(x);
41  }
42
43  void splay(int x)
44  {
45      while (a[x].fa)
46      {
47          int y = a[x].fa, z = a[y].fa;
48          if (z)
49          {
50              bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
51              rotate(same ? y : x);
52          }
53          rotate(x);
54      }
55  }
56
57  // Splay the last visited node even when the value does not exist.
58  int find(ll value)
59  {
60      int x = root, last = 0;
61      while (x && a[x].val != value)
62      {
63          last = x;

```

```

64         x = a[x].ch[value > a[x].val];
65     }
66     if (x)
67         splay(x);
68     else if (last)
69         splay(last);
70     return x;
71 }
72
73 void insert(ll value)
74 {
75     int x = root, parent = 0;
76     while (x && a[x].val != value)
77     {
78         parent = x;
79         x = a[x].ch[value > a[x].val];
80     }
81     if (x)
82     {
83         ++a[x].cnt;
84         pull(x);
85     }
86     else
87     {
88         x = node(value, parent);
89         if (parent)
90             a[parent].ch[value > a[parent].val] = x;
91         else
92             root = x;
93     }
94     splay(x);
95 }
96
97 bool erase(ll value)
98 {
99     int x = find(value);
100    if (!x) return false;
101    if (a[x].cnt > 1)
102    {
103        --a[x].cnt;
104        pull(x);
105        return true;
106    }
107    int l = a[x].ch[0], r = a[x].ch[1];
108    if (l) a[l].fa = 0;
109    if (r) a[r].fa = 0;
110    if (!l)
111    {
112        root = r;
113        return true;
114    }
115    root = l;
116    x = l;
117    while (a[x].ch[1]) x = a[x].ch[1];
118    splay(x);
119    a[x].ch[1] = r;

```

```

120     if (r) a[r].fa = x;
121     pull(x);
122     return true;
123 }
124
125 int rank(ll value)
126 {
127     int x = root, last = 0, ans = 1;
128     while (x)
129     {
130         last = x;
131         if (value <= a[x].val)
132             x = a[x].ch[0];
133         else
134         {
135             ans += a[a[x].ch[0]].siz + a[x].cnt;
136             x = a[x].ch[1];
137         }
138     }
139     if (last) splay(last);
140     return ans;
141 }
142
143 ll kth(int k)
144 {
145     assert(1 <= k && k <= size());
146     int x = root;
147     while (true)
148     {
149         int left = a[a[x].ch[0]].siz;
150         if (k <= left)
151             x = a[x].ch[0];
152         else if (k <= left + a[x].cnt)
153         {
154             splay(x);
155             return a[x].val;
156         }
157         else
158         {
159             k -= left + a[x].cnt;
160             x = a[x].ch[1];
161         }
162     }
163 }
164
165 optional<ll> prev(ll value)
166 {
167     int x = root, last = 0, answer = 0;
168     while (x)
169     {
170         last = x;
171         if (a[x].val < value)
172         {
173             answer = x;
174             x = a[x].ch[1];
175         }

```

```
176         else
177             x = a[x].ch[0];
178     }
179     if (last) splay(last);
180     if (!answer) return nullopt;
181     splay(answer);
182     return a[answer].val;
183 }
184
185 optional<ll> next(ll value)
186 {
187     int x = root, last = 0, answer = 0;
188     while (x)
189     {
190         last = x;
191         if (a[x].val > value)
192         {
193             answer = x;
194             x = a[x].ch[0];
195         }
196         else
197             x = a[x].ch[1];
198     }
199     if (last) splay(last);
200     if (!answer) return nullopt;
201     splay(answer);
202     return a[answer].val;
203 }
204 };
```

第六章 可持久化数据结构

6.1 可持久化线段树

6.1.1 主席树：静态区间第 k 小

构造传入 0-based 数组， $kth(l,r,k)$ 的区间使用 1-based 闭区间；要求 $1 \leq k \leq r-l+1$ 。离散化后每个前缀一个版本，单次 $O(\log n)$ ，总空间 $O(n \log n)$ 。

```
1 struct PersistentKth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     vector<Node> t{Node{}};
9     vector<int> root{0};
10    vector<long long> vals;
11
12    PersistentKth(const vector<long long> &a) : vals(a)
13    {
14        sort(vals.begin(), vals.end());
15        vals.erase(unique(vals.begin(), vals.end()), vals.end());
16        for (long long x : a)
17        {
18            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
19            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
20        }
21    }
22
23    int add(int old, int l, int r, int x)
24    {
25        int p = t.size();
26        Node copy = t[old];
27        t.push_back(copy);
28        ++t[p].sum;
29        if (l < r)
30        {
31            int m = (l + r) / 2;
32            if (x <= m)
33            {
34                int v = add(t[old].l, l, m, x);
35                t[p].l = v;
36            }
37            else
38            {
39                int v = add(t[old].r, m + 1, r, x);
40                t[p].r = v;
41            }
42        }
43        return p;
44    }
```

```

44     }
45
46     long long kth(int l, int r, int k) const
47     {
48         assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49         int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50         while (L < R)
51         {
52             int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53             if (k <= c)
54             {
55                 u = t[u].l;
56                 v = t[v].l;
57                 R = m;
58             }
59             else
60             {
61                 k -= c;
62                 u = t[u].r;
63                 v = t[v].r;
64                 L = m + 1;
65             }
66         }
67         return vals[L];
68     }
69 };

```

6.1.2 可持久化数组：历史版本分叉与单点修改

PersistentArray(a,changes=0), a 是非空 0-based 初值数组；第 0 版为初始状态。set(version,pos,value) 基于任意已有版本修改 1-based 位置，创建并返回新版本编号；copy(version) 以 $O(1)$ 时间复制版本；query(version,pos) 只读且不自动创建版本。values 支持 long long 全范围。每次修改复制根到叶的节点，旧版本不变；build 为内部函数，不可覆盖历史节点。建树 $O(n)$ ，修改及查询 $O(\log n)$ ，初始 $2n-1$ 个节点，每次修改至多 $\text{ceil}(\log_2(n))+1$ 个；总空间 $O(n + \text{修改数} * \log n + \text{版本数})$ 。changes 仅用于预留节点池，低估后可扩容，所有外部引用应使用下标，不保存 vector 元素引用。节点下标用 int，节点总量须可表示。

```

1 struct PersistentArray
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0;
8         ll value = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root;
14
15     PersistentArray(const vector<ll> &a, int changes = 0) : n(a.size())
16     {
17         assert(n > 0 && changes >= 0);
18         int levels = 1;
19         for (int x = n - 1; x; x >>= 1) levels++;

```

```

20     t.reserve(2ULL * n + 1ULL * changes * levels + 1);
21     root.push_back(build(1, n, a));
22 }
23
24 int build(int l, int r, const vector<ll> &a)
25 {
26     int p = t.size();
27     t.push_back(Node{});
28     if (l == r)
29         t[p].value = a[l - 1];
30     else
31     {
32         int m = (l + r) / 2;
33         int left = build(l, m, a);
34         int right = build(m + 1, r, a);
35         t[p].l = left;
36         t[p].r = right;
37     }
38     return p;
39 }
40
41 int modify(int old, int l, int r, int pos, ll value)
42 {
43     int p = t.size();
44     Node copy = t[old];
45     t.push_back(copy);
46     if (l == r)
47         t[p].value = value;
48     else
49     {
50         int m = (l + r) / 2;
51         if (pos <= m)
52         {
53             int child = modify(t[old].l, l, m, pos, value);
54             t[p].l = child;
55         }
56         else
57         {
58             int child = modify(t[old].r, m + 1, r, pos, value);
59             t[p].r = child;
60         }
61     }
62     return p;
63 }
64
65 // Create a new version from any existing version. Positions are 1-based.
66 int set(int version, int pos, ll value)
67 {
68     assert(0 <= version && version < (int)root.size());
69     assert(1 <= pos && pos <= n);
70     int p = modify(root[version], 1, n, pos, value);
71     root.push_back(p);
72     return (int)root.size() - 1;
73 }
74
75 int copy(int version)

```

```

76     {
77         assert(0 <= version && version < (int)root.size());
78         root.push_back(root[version]);
79         return (int)root.size() - 1;
80     }
81
82     ll query(int version, int pos) const
83     {
84         assert(0 <= version && version < (int)root.size());
85         assert(1 <= pos && pos <= n);
86         int p = root[version], l = 1, r = n;
87         while (l < r)
88         {
89             int m = (l + r) / 2;
90             if (pos <= m)
91             {
92                 p = t[p].l;
93                 r = m;
94             }
95             else
96             {
97                 p = t[p].r;
98                 l = m + 1;
99             }
100         }
101         return t[p].value;
102     }
103 };

```

6.1.3 可持久化区间加与版本拼接

PersistentRange(n) 建立长度 n 的全零版本 0, $n \geq 1$; 使用隐式节点, 不预建整棵树。add(v, l, r, x) 在版本 v 的 $[l, r]$ 加 x ; splice(dst, src, l, r) 将 dst 的 $[l, r]$ 替换为 src 同一位置的值, 其他位置保留 dst; 二者均返回新版本编号。query(v, l, r) 返回区间和, const 且不分配节点。下标 0-based、区间半开, 允许空区间; 空修改及零增量仍创建版本但复用根, 整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先, 用 offset 修正跨版本拼接; 每次修改/拼接/查询 $O(\log n)$, 版本别名 $O(1)$, 总节点 $O(q \log n)$ 。sum、lazy、delta、carry、offset 及其中间和差和长度乘积均须在 long long 内; 节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根, 不要外部改写。

```

1 struct PersistentRange
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int left = 0, right = 0;
8         ll sum = 0, lazy = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root{0};
14
15     explicit PersistentRange(int size) : n(size) { assert(n > 0); }
16

```

```
17  int clone(int p)
18  {
19      Node node = t[p];
20      t.push_back(node);
21      return int(t.size()) - 1;
22  }
23
24  void pull(int p, int len)
25  {
26      t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
27  }
```

区间加与版本拼接（接上页同一结构体）：

```

29  int change(int p, int l, int r, int ql, int qr, ll value)
30  {
31      if (qr <= l || r <= ql) return p;
32      int u = clone(p);
33      if (ql <= l && r <= qr)
34      {
35          t[u].lazy += value;
36          t[u].sum += value * (r - l);
37          return u;
38      }
39      int m = l + (r - l) / 2;
40      int a = change(t[p].left, l, m, ql, qr, value);
41      int b = change(t[p].right, m, r, ql, qr, value);
42      t[u].left = a;
43      t[u].right = b;
44      pull(u, r - l);
45      return u;
46  }
47
48  int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
49  {
50      if (qr <= l || r <= ql) return p;
51      if (ql <= l && r <= qr)
52      {
53          if (offset == 0) return q;
54          int u = clone(q);
55          t[u].lazy += offset;
56          t[u].sum += offset * (r - l);
57          return u;
58      }
59      int u = clone(p), m = l + (r - l) / 2;
60      offset += t[q].lazy - t[p].lazy;
61      int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
62      int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
63      t[u].left = a;
64      t[u].right = b;
65      pull(u, r - l);
66      return u;
67  }

```

只读查询与对外接口（接上页同一结构体）：

```

69  ll get(int p, int l, int r, int ql, int qr, ll carry) const
70  {
71      if (qr <= l || r <= ql) return 0;
72      if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
73      carry += t[p].lazy;
74      int m = l + (r - l) / 2;
75      return get(t[p].left, l, m, ql, qr, carry) +
76             get(t[p].right, m, r, ql, qr, carry);
77  }
78
79  void check(int version, int l, int r) const
80  {
81      assert(0 <= version && version < int(root.size()));
82      assert(0 <= l && l <= r && r <= n);
83  }
84
85  int add(int version, int l, int r, ll value)
86  {
87      check(version, l, r);
88      int p = l == r || value == 0 ? root[version]
89              : change(root[version], 0, n, l, r, value);
90      root.push_back(p);
91      return int(root.size()) - 1;
92  }
93
94  int splice(int destination, int source, int l, int r)
95  {
96      check(destination, l, r);
97      check(source, l, r);
98      int p = l == r ? root[destination]
99              : replace(root[destination], root[source], 0, n, l, r, 0);
100     root.push_back(p);
101     return int(root.size()) - 1;
102 }
103
104 ll query(int version, int l, int r) const
105 {
106     check(version, l, r);
107     return l == r ? 0 : get(root[version], 0, n, l, r, 0);
108 }
109 };

```

6.1.4 树上路径点权第 k 小

TreePathKth(a) 传入非空 0-based 权值数组，节点编号 1..n；add(u,v) 加无向边，要求恰好构成一棵树。build(root=1) 递归 DFS 建立根到各节点的频数版本与倍增表，可换根重建，旧版本自动清空。kth(u,v,k) 返回简单路径上第 k 小的点权，包含两个端点，每个节点计一次； $1 \leq k \leq$ 路径点数。lca(u,v) 返回当前预处理根下的最近公共祖先。使用 $\text{root}[u] + \text{root}[v] - \text{root}[\text{lca}] - \text{root}[\text{parent}(\text{lca})]$ ，编号 0 为全零版本，避免少算或重复算 LCA。支持重复值与完整 long long 权值，内部只统计频数；权值和树结构在查询阶段保持不变。预处理 $O(n \log n)$ ，每次查询 $O(\log n)$ ，空间 $O(n \log n)$ 。递归深度最坏 $O(n)$ ，保持递归写法。

```

1 struct TreePathKth
2 {

```

```

3      using ll = long long;
4
5      struct Node
6      {
7          int l = 0, r = 0, sum = 0;
8      };
9
10     int n, lg;
11     vector<vector<int>> g, up;
12     vector<int> root, dep;
13     vector<ll> value, vals;
14     vector<Node> t;
15
16     TreePathKth(const vector<ll> &a)
17         : n(a.size()), g(n + 1), root(n + 1), dep(n + 1), value(a)
18     {
19         assert(n > 0);
20         lg = 1;
21         while ((1LL << lg) <= n) lg++;
22         up.assign(lg, vector<int>(n + 1));
23     }
24
25     void add(int u, int v)
26     {
27         g[u].push_back(v);
28         g[v].push_back(u);
29     }
30
31     int insert(int old, int l, int r, int x)
32     {
33         int p = t.size();
34         Node copy = t[old];
35         t.push_back(copy);
36         t[p].sum++;
37         if (l == r) return p;
38         int m = (l + r) / 2;
39         if (x <= m)
40         {
41             int child = insert(t[old].l, l, m, x);
42             t[p].l = child;
43         }
44         else
45         {
46             int child = insert(t[old].r, m + 1, r, x);
47             t[p].r = child;
48         }
49         return p;
50     }
51
52     void dfs(int u, int p)
53     {
54         up[0][u] = p;
55         for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
56         int x = lower_bound(vals.begin(), vals.end(), value[u - 1]) - vals.begin();
57         root[u] = insert(root[p], 0, (int)vals.size() - 1, x);
58         for (int v : g[u])

```

```

59     {
60         if (v == p) continue;
61         dep[v] = dep[u] + 1;
62         dfs(v, u);
63     }
64 }
65
66 void build(int r = 1)
67 {
68     assert(1 <= r && r <= n);
69     vals = value;
70     sort(vals.begin(), vals.end());
71     vals.erase(unique(vals.begin(), vals.end()), vals.end());
72     t.assign(1, Node{});
73     dep[r] = 0;
74     dfs(r, 0);
75 }
76
77 int lca(int u, int v) const
78 {
79     if (dep[u] < dep[v]) swap(u, v);
80     int diff = dep[u] - dep[v];
81     for (int j = 0; j < lg; j++)
82         if (diff >> j & 1) u = up[j][u];
83     if (u == v) return u;
84     for (int j = lg - 1; j >= 0; j--)
85         if (up[j][u] != up[j][v])
86             {
87                 u = up[j][u];
88                 v = up[j][v];
89             }
90     return up[0][u];
91 }
92
93 ll kth(int u, int v, int k) const
94 {
95     int z = lca(u, v);
96     assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
97     int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];
98     int l = 0, r = (int)vals.size() - 1;
99     while (l < r)
100     {
101         int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
102         int m = (l + r) / 2;
103         if (k <= count)
104             {
105                 a = t[a].l;
106                 b = t[b].l;
107                 c = t[c].l;
108                 d = t[d].l;
109                 r = m;
110             }
111         else
112             {
113                 k -= count;

```

```

114         a = t[a].r;
115         b = t[b].r;
116         c = t[c].r;
117         d = t[d].r;
118         l = m + 1;
119     }
120 }
121 return vals[l];
122 }
123 };

```

6.1.5 静态区间不同数计数（前驱位置主席树）

PersistentDistinct(a) 传入非空 0-based 数组，query(l,r) 返回闭区间内不同值的个数， $1 \leq l \leq r \leq n$ 。支持重复值与完整 long long 值域；建成后不支持改值，查询顺序任意。令 prev[i] 为同值上次出现的位置，未出现为 0；root[i] 在值域 $0..n-1$ 中统计前 i 个 prev。区间内每种值恰有一次 prev[i]<l，因此答案为 root[r]-root[l-1] 中位置 $0..l-1$ 的频数和。每个元素只复制一条节点路径，预处理 $O(n \log n)$ ，查询 $O(\log n)$ ，空间 $O(n \log n)$ 。动态版预留 $n \cdot (\text{ceil}(\log_2(n))+1)+1$ 个节点，避免扩容峰值；root 和节点池为内部数据。

```

1 struct PersistentDistinct
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     int n;
9     vector<Node> t{Node{}};
10    vector<int> root;
11
12    PersistentDistinct(const vector<long long> &a) : n(a.size()), root(n + 1)
13    {
14        assert(n > 0);
15        int height = 1;
16        while ((1LL << (height - 1)) < n) height++;
17        t.reserve(size_t(n) * height + 1);
18        vector<long long> vals = a;
19        sort(vals.begin(), vals.end());
20        vals.erase(unique(vals.begin(), vals.end()), vals.end());
21        vector<int> last(vals.size());
22        for (int i = 1; i <= n; i++)
23        {
24            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
25            root[i] = insert(root[i - 1], 0, n - 1, last[x]);
26            last[x] = i;
27        }
28    }
29
30    int insert(int old, int l, int r, int x)
31    {
32        int p = t.size();
33        Node copy = t[old];
34        t.push_back(copy);
35        t[p].sum++;
36        if (l < r)

```

```

37     {
38         int m = (l + r) / 2;
39         if (x <= m)
40         {
41             int child = insert(t[old].l, l, m, x);
42             t[p].l = child;
43         }
44         else
45         {
46             int child = insert(t[old].r, m + 1, r, x);
47             t[p].r = child;
48         }
49     }
50     return p;
51 }
52
53 int query(int l, int r) const
54 {
55     assert(1 <= l && l <= r && r <= n);
56     int u = root[l - 1], v = root[r];
57     int L = 0, R = n - 1, answer = 0;
58     while (L < R)
59     {
60         int m = (L + R) / 2;
61         if (l - 1 <= m)
62         {
63             u = t[u].l;
64             v = t[v].l;
65             R = m;
66         }
67         else
68         {
69             answer += t[t[v].l].sum - t[t[u].l].sum;
70             u = t[u].r;
71             v = t[v].r;
72             L = m + 1;
73         }
74     }
75     return answer + t[v].sum - t[u].sum;
76 }
77 };

```

第七章 树套树

7.1 树状数组套权值线段树

7.1.1 动态区间第 k 小（树状数组套权值线段树）

DynamicKth(a,future) 传入非空 0-based 初值, future 包含所有将来可能赋入的新值; 内部自动合并初值并离散化。set(pos,value) 单点赋值, kth(l,r,k) 查询闭区间第 k 小, 下标均为 1-based; $1 \leq k \leq r-l+1$ 。允许重复值与完整 long long 权值; 新值必须已在离散化集合内, 操作可按原顺序执行, 不保留历史版本。每个树状数组节点对应一棵可变权值线段树; 删除旧值后插入新值。空节点回收到 free_head 链, 重复修改不会按操作次数累计泄漏节点。设 D 为不同值数, $S=n+\text{future.size()}$, 预处理 $O(S \log S+n \log n \log(D+1))$, 单次修改及查询 $O(\log n \log(D+1))$ 。节点数上界 $B=(\text{ceil}(\log_2(D))+1) \cdot \sum \text{lowbit}(i)$, 求和 $i=1..n$; 空间 $O(S+n+B)$ 。update、modify、id 和节点池均为内部接口; 不能手动减去不存在的值。

```
1 struct DynamicKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, free_head = 0;
11    vector<int> bit, rank;
12    vector<ll> vals;
13    vector<Node> t{Node{}};
14
15    DynamicKth(const vector<ll> &a, vector<ll> future = {})
16        : n(a.size()), bit(n + 1), rank(n + 1)
17    {
18        assert(n > 0);
19        vals = future;
20        vals.insert(vals.end(), a.begin(), a.end());
21        sort(vals.begin(), vals.end());
22        vals.erase(unique(vals.begin(), vals.end()), vals.end());
23        for (int i = 1; i <= n; i++)
24        {
25            rank[i] = id(a[i - 1]);
26            modify(i, rank[i], 1);
27        }
28    }
29
30    int id(ll value) const
31    {
32        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
33        assert(p < (int)vals.size() && vals[p] == value);
34        return p;
35    }
36
37    int new_node()
38    {
```

```

39     if (!free_head)
40     {
41         t.push_back(Node{});
42         return (int)t.size() - 1;
43     }
44     int p = free_head;
45     free_head = t[p].l;
46     t[p] = Node{};
47     return p;
48 }
49
50 int update(int p, int l, int r, int x, int delta)
51 {
52     if (!p) p = new_node();
53     t[p].sum += delta;
54     if (l < r)
55     {
56         int m = (l + r) / 2;
57         if (x <= m)
58         {
59             int child = update(t[p].l, l, m, x, delta);
60             t[p].l = child;
61         }
62         else
63         {
64             int child = update(t[p].r, m + 1, r, x, delta);
65             t[p].r = child;
66         }
67     }
68     if (!t[p].sum)
69     {
70         t[p].l = free_head;
71         free_head = p;
72         return 0;
73     }
74     return p;
75 }
76
77 void modify(int pos, int x, int delta)
78 {
79     for (int i = pos; i <= n; i += i & -i)
80         bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
81 }

```

```

83 void set(int pos, ll value)
84 {
85     assert(1 <= pos && pos <= n);
86     int x = id(value);
87     if (rank[pos] == x) return;
88     modify(pos, rank[pos], -1);
89     modify(pos, x, 1);
90     rank[pos] = x;
91 }
92
93 ll kth(int l, int r, int k) const
94 {
95     assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
96     vector<int> a, b;
97     for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
98     for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
99     int L = 0, R = (int)vals.size() - 1;
100    while (L < R)
101    {
102        int count = 0;
103        for (int p : a) count += t[t[p].l].sum;
104        for (int p : b) count -= t[t[p].l].sum;
105        bool left = k <= count;
106        if (!left) k -= count;
107        for (int &p : a) p = left ? t[p].l : t[p].r;
108        for (int &p : b) p = left ? t[p].l : t[p].r;
109        int m = (L + R) / 2;
110        if (left)
111            R = m;
112        else
113            L = m + 1;
114    }
115    return vals[L];
116 }
117 };

```

第八章 动态树

8.1 Link Cut Tree

8.1.1 Link-cut tree: 动态森林与路径异或

构造 LinkCutTree(n), 点号 1..n, 初始权值为零。set(x,value) 修改点权; link(x,y) 仅在不同连通块之间连边并返回是否成功; cut(x,y) 只删除真实存在的边, 否则返回 false。connected 查询连通性; query(x,y) 返回路径点权异或, 要求两点连通, 支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 $O(\log n)$, $O(n)$ 空间, splay 下传使用显式栈, 支持长链。权值为 unsigned long long。仅维护路径异或, 不包含虚子树聚合或动态直径。

```
1 struct LinkCutTree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    vector<Node> a;
13    vector<int> st;
14
15    LinkCutTree(int n) : a(n + 1), st(n + 1) {}
16
17    bool is_root(int x) const
18    {
19        int f = a[x].fa;
20        return a[f].ch[0] != x && a[f].ch[1] != x;
21    }
22
23    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }
24
25    void reverse(int x)
26    {
27        if (!x) return;
28        swap(a[x].ch[0], a[x].ch[1]);
29        a[x].rev = !a[x].rev;
30    }
31
32    void push(int x)
33    {
34        if (!a[x].rev) return;
35        reverse(a[x].ch[0]);
36        reverse(a[x].ch[1]);
37        a[x].rev = false;
38    }
```

```
40 void rotate(int x)
41 {
42     int y = a[x].fa, z = a[y].fa;
43     int k = a[y].ch[1] == x;
44     int w = a[x].ch[k ^ 1];
45     if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
46     a[x].fa = z;
47     a[y].ch[k] = w;
48     if (w) a[w].fa = y;
49     a[x].ch[k ^ 1] = y;
50     a[y].fa = x;
51     pull(y);
52     pull(x);
53 }
54
55 void splay(int x)
56 {
57     int top = 0, y = x;
58     st[top++] = y;
59     while (!is_root(y))
60     {
61         y = a[y].fa;
62         st[top++] = y;
63     }
64     while (top) push(st[--top]);
65     while (!is_root(x))
66     {
67         int y = a[x].fa, z = a[y].fa;
68         if (!is_root(y))
69         {
70             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
71             rotate(same ? y : x);
72         }
73         rotate(x);
74     }
75 }
```

```
77 void access(int x)
78 {
79     int last = 0;
80     for (int y = x; y; y = a[y].fa)
81     {
82         splay(y);
83         // The old right child's fa remains y: it becomes a path-parent.
84         a[y].ch[1] = last;
85         if (last) a[last].fa = y;
86         pull(y);
87         last = y;
88     }
89     splay(x);
90 }
91
92 void make_root(int x)
93 {
94     access(x);
95     reverse(x);
96 }
97
98 int find_root(int x)
99 {
100     access(x);
101     push(x);
102     while (a[x].ch[0])
103     {
104         x = a[x].ch[0];
105         push(x);
106     }
107     splay(x);
108     return x;
109 }
```

```
111     bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }
112
113     bool link(int x, int y)
114     {
115         make_root(x);
116         if (find_root(y) == x) return false;
117         a[x].fa = y;
118         return true;
119     }
120
121     bool cut(int x, int y)
122     {
123         make_root(x);
124         access(y);
125         if (a[y].ch[0] != x || a[x].ch[1]) return false;
126         a[y].ch[0] = 0;
127         a[x].fa = 0;
128         pull(y);
129         return true;
130     }
131
132     void set(int x, U value)
133     {
134         access(x);
135         a[x].val = value;
136         pull(x);
137     }
138
139     // x and y must be connected. This is a path aggregate, not a subtree sum.
140     U query(int x, int y)
141     {
142         make_root(x);
143         access(y);
144         return a[y].sum;
145     }
146 };
```

第九章 配套知识与验证范围

9.1 可持久化区间拼接的懒标记

代码见第 35 页。对象从全零版本开始，同一对象内版本共享坐标域。区间加仅复制被访问节点，不向旧子节点下传标记。节点的和满足 $\text{sum} = \text{left.sum} + \text{right.sum} + \text{lazy} \cdot \text{len}$ 。读取时把祖先累积增量作为参数传下去，因此查询完全只读。

拼接含义为将目标版本某个半开区间替换成源版本同一坐标区间。由于两棵树祖先的 lazy 可能不同，不能直接将源子树挂到目标祖先下面。递归参数 offset 保持“源祖先增量减目标祖先增量”；下降时更新为 $\text{offset} + \text{source.lazy} - \text{destination.lazy}$ 。当区间完整覆盖，复制源节点并将这个 offset 加到其 lazy 和 sum，此后挂到目标祖先下面时，逻辑值才与源版本相同。offset 为零时直接共享源节点；没有覆盖时保留目标节点。空区间、零增量与全区间拼接都可仅增加版本别名。

每一层至多有两个边界节点，其他分支直接返回或复制一个整段节点，所以单次修改、拼接均为 $O(\log n)$ 新节点，查询为 $O(\log n)$ 时间、零新节点。不支持不同坐标偏移的平移粘贴，也不回收仍被旧版本引用的节点。所有 long long 的和、差、懒标记与长度乘积都应在范围内；节点数与版本数须小于 INT_MAX。

验证用独立向量保存各分支版本，逐区间检查更新、拼接和历史版本；包含正负增量、不同祖先标记、空区间、全段共享，以及查询前后的节点数量检查。规模验证包含长度 200000 的 10000 次区间版本组合，并在长度 INT_MAX 的隐式坐标域验证端点操作与共享根。普通与 ASan/UBSan 均通过；区间加 1、后缀拼接和单点读取的应用已获 QOJ8240 双 AC，其余通用接口范围仍由本地验证覆盖。来源需求是2023 杭州站题解 第 24-26 页的 K 题，完整驱动及验证见下节。

9.2 杭州 2023 K：跨版本拼接应用

[Card Game](#) 的完整驱动在 `verify/qoj/8240/*.cpp`，使用前节的可持久化区间模板。将下标转为从 0 开始，设 $F_i(r)$ 为区间 $[i, r]$ 的剩余牌数， p 为 i 之后第一张与 a_i 相同的牌，若不存在则 $p = n$ 。当 $r < p$ 时，第一张牌始终保留，剩余牌的过程与从 $i + 1$ 开始相同；当 $r \geq p$ 时，到达 p 会将此前区间全部清空。因此

$$F_i(r) = \begin{cases} 1 + F_{i+1}(r), & r < p, \\ F_{p+1}(r), & r \geq p. \end{cases}$$

每个版本的坐标是右端点 r ，值是 $F_i(r)$ 。从 $i = n - 1$ 向前处理：复制版本 $i + 1$ 并在 $[i, p)$ 加 1，再把 $[p, n)$ 拼接自版本 $p + 1$ 。 $p = n$ 时省去后一步。询问原问题的 $[l, r]$ 时，从版本 $l - 1$ 读取坐标 $r - 1$ 的单点值。端点先与上次答案异或解码，不能将输入查询预先离线排序。

$n, q \leq 300000$ ，牌号位于 $[1, n]$ 。构建及查询时间分别为 $O(n \log n)$ 、 $O(q \log n)$ 。驱动另计算各次操作可能访问的相交节点数作为分配上界，一次 reserve 足够容纳整个构建过程，避免复制扩容的临时峰值。该计数仅依据区间结构，不影响牌局答案，且同样为 $O(n \log n)$ 。

本地参照按题意逐张放牌，遇重复就实际删除栈中相应后缀。穷举长度不超过 7 的二元牌序列与全部区间，并打乱询问顺序、真实编码异或输入。另有随机牌局、 $n = q = 300000$ 的全异、全同、周期序列闭式检查，以及稀疏重复大数组的短区间逐张模拟。两套驱动均在线 AC，各 41 个正式测试和 3 个额外测试全过：[动态版](#)最慢 789 ms、452688 KB，[传统版](#)最慢 752 ms、452492 KB。此应用仅调用区间加 1、后缀拼接、单点读取；一般有符号增量与区间求和仍由前节本地测试支持。

9.3 并查集模板题的接口范围

[P3367](#) 输入点号为 1-based，驱动统一减一后调用 dsu。[记录 297668102](#) 通过全部 20 个测试，最慢单点 196 ms、峰值内存 2.27 MB。该输出验证 merge、same 及其调用的 find，不验证 size、groups；后两者已由独立标签参考和 50 万点长链测试核对。

第一部分

跨类依赖

附录 A 数论

A.1 模算术简介

A.1.1 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且 ≥ 2 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。支持 `+=`、`-=`、`*=`、`/=`，返回自身引用，允许自赋值和链式运算；可作为 Fenwick 区间和的值类型。复合除法沿用非零元素、素数模数前提。

```
1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
6
7     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
8
9     ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
10
11    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
12
13    ModInt pow(long long e) const
14    {
15        assert(e >= 0);
16        ModInt a = *this, r = 1;
17        for (; e >>= 1, a = a * a)
18            if (e & 1) r = r * a;
19        return r;
20    }
21
22    ModInt inv() const
23    {
24        assert(v);
25        return pow(mod - 2);
26    } // prime modulus
27
28    ModInt operator/(ModInt b) const { return *this * b.inv(); }
29
30    ModInt &operator+=(ModInt b) { return *this = *this + b; }
31
32    ModInt &operator-=(ModInt b) { return *this = *this - b; }
33
34    ModInt &operator*=(ModInt b) { return *this = *this * b; }
35
36    ModInt &operator/=(ModInt b) { return *this = *this / b; }
37 };
```

算法索引

AffineSegTree, 13

cc_hash_table, 1

dsu, 2

DynamicKth, 43

Fenwick, 4

GcdSequenceTreap, 24

gp_hash_table, 1

gp_map, 1

lazy_segtree, 5

LazySeg, 12

LiChao, 15

LinkCutTree, 46

ModInt, 54

OrderedSplay, 27

OrderedTreap, 18

ost, 18

PersistentArray, 33

PersistentDistinct, 41

PersistentKth, 32

PersistentRange, 35

RollbackDSU, 3

rp, 17

segtree, 8

SequenceTreap, 21

TreePathKth, 38